

Tracking Performance Drift in Large Language Models Across Successive Version Updates

A Review of Measurement, Mechanisms, and Mitigation in Longitudinal LLM Evaluation

Abstract

Large language models (LLMs) deployed as commercial application programming interfaces (APIs) are routinely updated without formal versioning guarantees, and empirical studies show that these updates can produce substantial, unannounced changes in model behavior and task accuracy — a phenomenon this paper terms LLM performance drift. Building on longitudinal audits of GPT-3.5 and GPT-4 (Chen, Zaharia, & Zou, 2024), backward-compatibility research in classical machine learning (Bansal et al., 2019; Yan et al., 2021), and the concept-drift and catastrophic-forgetting literatures (Gama, Žliobaitė, Bifet, Pechenizkiy, & Bouchachia, 2014; Kirkpatrick et al., 2017), this paper synthesizes evidence on how, why, and to what extent LLM performance drifts across successive version updates. We characterize four interacting drift dimensions — capability drift, behavioral and stylistic drift, alignment and safety drift, and instance-level regression (“negative flips”) — and review current approaches to detecting and mitigating them, including longitudinal benchmarking systems, holistic evaluation frameworks, backward-compatible training, and model/system-card documentation. We then identify methodological challenges that complicate drift measurement, including API opacity, benchmark data contamination, sampling non-determinism, and the absence of standardized drift metrics. We conclude by outlining research gaps — a shared drift-reporting standard, causal attribution methods separating data, architecture, and alignment effects, and continuous evaluation-as-infrastructure — and argue that systematic drift tracking should become a routine component of responsible LLM deployment rather than an ad hoc research exercise.

Keywords

large language models; performance drift; model versioning; concept drift; catastrophic forgetting; alignment tax; benchmark contamination; regression testing; negative flips; longitudinal evaluation

1. Introduction

Large language models such as GPT-4, Claude, Gemini, and Llama are no longer static artifacts released once and left unchanged. Commercial LLM providers routinely issue silent or lightly documented updates — changes to model weights, decoding parameters, safety filters, or the reinforcement-learning-from-human-feedback (RLHF) policy underlying a deployed endpoint — while keeping the externally visible model name constant (e.g., “gpt-4” or “gpt-3.5-turbo”). Because these updates are frequently opaque to end users and researchers alike, a model that behaves acceptably on a given task at one point in time may behave very differently on the same task months later, even though the interface and nominal identity of the model have not changed. This paper is concerned with this

phenomenon, which we refer to as performance drift: measurable change, positive or negative, in an LLM's accuracy, consistency, or behavior across successive versions of what is nominally “the same” deployed model.

The empirical case for taking LLM drift seriously is now well established. In one of the first systematic longitudinal audits of a commercial LLM service, Chen, Zaharia, and Zou (2024) evaluated the March 2023 and June 2023 snapshots of GPT-3.5 and GPT-4 on math problem solving, code generation, sensitive-question answering, and visual reasoning, and found that accuracy on some tasks swung dramatically between the two snapshots — for example, GPT-4's accuracy at identifying prime versus composite numbers fell from roughly 97.6% to 2.4% over the three-month interval, while GPT-3.5's accuracy on the same task rose substantially. Complementary work using the ChatLog framework recorded ChatGPT's responses to a fixed battery of natural language understanding and generation benchmarks on a rolling monthly and daily basis and likewise documented measurable, if generally smaller, behavioral shifts over time (Tu et al., 2023). These findings matter because a large and growing ecosystem of downstream applications, evaluation pipelines, and academic studies quietly assumes that an LLM identified by a fixed name is a fixed artifact — an assumption that longitudinal audits show to be false.

Performance drift in LLMs sits at the intersection of several previously distinct research traditions. The machine learning systems literature has long studied “negative flips” and backward compatibility in the context of periodically retrained production classifiers (Bansal et al., 2019; Yan et al., 2021; Xie, Lai, Xiong, Zhang, & Soatto, 2021), where an updated model that improves aggregate accuracy nonetheless makes new mistakes on inputs the previous model handled correctly. The streaming-data literature has formalized concept drift as a change over time in the joint distribution relating inputs to outputs (Gama et al., 2014). The continual learning literature has documented catastrophic forgetting, in which sequential training on new objectives erodes previously acquired capabilities (McCloskey & Cohen, 1989; Kirkpatrick et al., 2017). And the LLM alignment literature has separately documented an “alignment tax” — measurable capability degradation on standard benchmarks that can accompany RLHF fine-tuning (Ouyang et al., 2022) — as well as emergent behavioral pathologies such as sycophancy that appear to intensify with model scale and instruction tuning (Perez et al., 2023; Sharma et al., 2023; Wei, Huang, Lu, Zhou, & Le, 2023). LLM performance drift is best understood as the point where these strands converge: a production LLM is simultaneously a periodically retrained classifier, a streaming-data learner, a continual learner, and an RLHF-aligned agent, and each of these framings contributes distinct mechanisms and distinct measurement tools relevant to understanding why its behavior changes across versions.

This paper makes three contributions. First, it synthesizes evidence from machine learning systems research, LLM-specific longitudinal audits, and alignment research into a unified taxonomy of drift dimensions relevant to LLMs. Second, it reviews the current landscape of approaches used to detect, quantify, and mitigate this drift, spanning longitudinal benchmarking platforms, holistic evaluation suites, backward-compatible training objectives, and documentation standards such as model cards. Third, it identifies the principal methodological obstacles to reliable drift measurement and proposes directions for a more standardized, infrastructure-level approach to tracking LLM performance over time.

The remainder of the paper is organized as follows: Section 2 reviews related work; Section 3 develops a technical taxonomy of drift mechanisms; Section 4 surveys current detection and mitigation approaches; Section 5 discusses research challenges and limitations; Section 6 outlines research gaps and future directions; and Section 7 concludes.

2. Background, Related Work, and Literature Review

2.1 Concept Drift and Model Regression in Classical Machine Learning

The idea that a deployed statistical model's performance can change over time predates LLMs by decades. Gama et al. (2014) define concept drift as a change in the joint probability distribution $P(X, Y)$ relating input features X to a target Y , and distinguish sudden, gradual, incremental, and recurring drift patterns, together with a taxonomy of detection methods based on monitoring error rates, data distributions, or model parameters over sliding windows. This framework was developed primarily for streaming classifiers such as fraud detectors or spam filters, where the environment generating the data genuinely changes. LLM drift differs in an important respect: rather than the external data distribution shifting under a fixed model, it is typically the model itself that changes — through retraining, fine-tuning, or reconfiguration — while the tasks users pose to it remain comparatively stable. This is closer to what the systems literature calls model update regression.

Bansal, Nushi, Kamar, Weld, Lasecki, and Horvitz (2019) were among the first to formalize this problem in human-AI interaction terms, showing that when a classifier is updated, users experience some errors as regressions even if aggregate accuracy improves, because the user has built a mental model of the system's prior behavior. They term instances that were previously classified correctly but are misclassified after an update “negative flips,” and show that minimizing negative flips can trade off against maximizing aggregate accuracy gains. Yan et al. (2021) extended this idea to image classification with “positive-congruent training,” an objective that explicitly constrains a new model to preserve the correct predictions of its predecessor. Xie et al. (2021) documented the same phenomenon in NLP model updates, showing that regression bugs are pervasive in production NLP pipelines and proposing measurement protocols for quantifying them. Schumann, Mansimov, Lai, Pappas, Gao, and Zhang (2024) proposed backward-compatible weight interpolation, which blends the weights of an old and a new model to reduce negative flips without sacrificing the new model's aggregate accuracy gains. This body of work establishes both the vocabulary (negative flip rate, positive-congruent training) and the core empirical finding — that model updates which improve aggregate metrics can still degrade performance on a non-trivial subset of previously well-handled inputs — that later LLM-specific studies would confirm at much larger scale.

2.2 Longitudinal Auditing of Commercial LLM Services

The most directly relevant empirical work is Chen, Zaharia, and Zou's (2024) longitudinal comparison of GPT-3.5 and GPT-4 snapshots collected in March and June 2023, published in the Harvard Data Science Review after initial circulation as an arXiv preprint. Beyond the headline finding on prime-number identification, the study documented drift in code generation (with the June 2023 version of GPT-4 producing code that was less frequently directly executable due to added non-code formatting), in

willingness to answer sensitive or opinion-eliciting questions, and in performance on multi-hop, knowledge-intensive question answering and visual reasoning tasks. The authors further hypothesized that a decline in instruction-following ability — rather than a decline in underlying reasoning capacity per se — could be a common factor behind several of the observed task-level drifts, since the June 2023 GPT-4 snapshot was substantially worse at following simple formatting instructions than its March counterpart. This instruction-following hypothesis is significant because it suggests that at least some LLM drift may originate in the alignment or fine-tuning layer of a model rather than in its core pretrained capabilities, a distinction with direct implications for how drift should be diagnosed and mitigated.

Complementary evidence comes from Tu et al.'s (2023) ChatLog framework, which records ChatGPT's responses to a fixed set of 21 natural language understanding and generation benchmarks on both monthly and daily cadences, framing the problem explicitly as a curve-fitting exercise over model outputs sampled at a sequence of time points. ChatLog reported generally smaller shifts than Chen et al. (2024) — often under five percent on aggregate benchmark scores — but confirmed that measurable behavioral change occurs even over short intervals and that some benchmarks are considerably more volatile than others. The discrepancy in magnitude between the two studies likely reflects differences in task selection: Chen et al. deliberately probed edge-case and instruction-sensitive tasks where drift is more visible, while ChatLog's broader benchmark suite averages over many tasks, some of which are comparatively drift-resistant. Taken together, these two studies indicate that drift magnitude is highly task-dependent, and that aggregate benchmark averages can understate the practical severity of drift on specific use cases.

2.3 Alignment Tax, Catastrophic Forgetting, and Sycophancy

A parallel literature, developed mostly independently of the longitudinal-auditing tradition, documents mechanisms by which the RLHF alignment process itself can degrade or alter LLM capabilities. Ouyang et al. (2022), in introducing InstructGPT, explicitly measured what they term an “alignment tax”: after RLHF fine-tuning, performance on public NLP benchmarks such as SQuAD and WMT translation tended to regress relative to the base pretrained model, an effect the authors partially mitigated by mixing pretraining-distribution gradient updates into the RLHF optimization (“PPO-ptx”). Because commercial providers periodically re-run or adjust their RLHF pipelines between model versions, alignment tax is a plausible mechanistic contributor to observed inter-version drift, distinct from changes in pretraining data or architecture. This connects to the older continual-learning literature on catastrophic forgetting, in which McCloskey and Cohen (1989) first documented that neural networks trained sequentially on new tasks can abruptly lose competence on earlier tasks, and Kirkpatrick et al. (2017) proposed elastic weight consolidation as a regularization-based mitigation that constrains updates to parameters identified as important for previously learned tasks via the Fisher information matrix. RLHF fine-tuning, applied repeatedly across model versions, can be viewed as a sequential learning process susceptible to the same forgetting dynamics that motivated this earlier work.

A further, behaviorally distinct axis of drift concerns sycophancy: the tendency of an LLM to tailor its responses toward a user's apparent opinions or expectations rather than toward accuracy. Perez et al. (2023) used model-written evaluations to show that larger and more heavily instruction-tuned language

models are systematically more likely to change a stated answer to match a user's expressed view. Sharma et al. (2023) extended this analysis to five production AI assistants and implicated human feedback data itself as a contributing cause, since human raters used to train preference models can systematically prefer agreeable but less accurate responses. Wei et al. (2023) confirmed that sycophancy occurs even on objectively verifiable claims (e.g., simple arithmetic) and proposed a synthetic-data fine-tuning intervention to reduce it. Because sycophancy intensity is plausibly sensitive to the exact RLHF policy and preference-model version in use, it is a natural candidate dimension of behavioral drift across model updates, distinct from, but potentially correlated with, accuracy-based capability drift.

2.4 Evaluation Infrastructure and Documentation Standards

A separate strand of work has focused not on characterizing drift directly but on building the evaluation infrastructure needed to detect it. Liang et al. (2022) introduced Holistic Evaluation of Language Models (HELM), which evaluates language models across a standardized taxonomy of scenarios and multiple metrics (accuracy, calibration, robustness, fairness, bias, toxicity, and efficiency) under fixed prompting conditions, explicitly intending HELM to function as a “living benchmark” that could, in principle, be re-run against successive model versions to expose exactly the kind of drift this paper is concerned with. On the documentation side, Mitchell et al. (2019) proposed model cards as a standardized reporting format covering a model's intended use, evaluation data, quantitative performance broken down by relevant subgroups, and known limitations; a model card explicitly designed to be reissued with each model version would provide a natural documentary trail for tracking drift, though as currently practiced in industry, model and system cards are not always updated with the granularity or frequency needed to support longitudinal comparison. Finally, the benchmark data contamination literature (Xu, Guan, Greene, & Kechadi, 2024) documents a confound that threatens both cross-sectional and longitudinal LLM evaluation alike: because newer model versions are frequently trained on more recent and more extensive web-scraped corpora, apparent performance improvements between versions can partly reflect benchmark memorization rather than genuine capability gains, complicating the attribution of any observed inter-version score change to real capability drift versus contamination-driven inflation.

3. A Technical Taxonomy of LLM Performance Drift

Synthesizing the literature reviewed above, we distinguish four interacting but analytically separable dimensions along which an LLM's behavior can drift across successive versions. These dimensions are not mutually exclusive causal mechanisms so much as distinct observable symptoms, any of which can, in a given case, be traced back to changes in pretraining data, model architecture, fine-tuning data, RLHF policy, decoding configuration, or system-level safety filtering.

3.1 Capability Drift

Capability drift refers to change in an LLM's accuracy or competence on well-defined tasks with objectively verifiable answers, such as arithmetic, code correctness, or factual question answering. This is the dimension most directly documented by Chen et al. (2024), whose prime-number and code-generation results show that capability drift can be large in magnitude, non-monotonic across versions (i.e., a later version is not guaranteed to be uniformly better than an earlier one on a given

task), and task-specific rather than global. Capability drift is the dimension most analogous to classical model regression as studied by Bansal et al. (2019) and Yan et al. (2021), and it is the dimension for which the negative-flip framework transfers most directly: a meaningful drift metric is not simply the change in aggregate accuracy between versions but the negative flip rate — the proportion of instances correctly handled by the prior version that are mishandled by the new one — since aggregate accuracy can remain stable or improve even as a substantial and disruptive subset of previously reliable behavior is lost. Echterhoff et al. (2024), in the MUSCLE study, quantified this directly for open-weight LLM families (Llama, Vicuna, Phi), finding negative flip rates of up to and over sixty percent on some downstream tasks when a fine-tuned adapter's underlying base model was updated, even though the fine-tuning procedure itself was held constant — demonstrating that capability drift in LLMs can propagate from a base model update into every downstream application built on top of it.

3.2 Behavioral and Stylistic Drift

Behavioral drift concerns changes in how an LLM communicates rather than whether it is correct: response length and verbosity, formatting conventions, tone, refusal phrasing, and adherence to explicit output-format instructions. Chen et al.'s (2024) observation that GPT-4's June 2023 snapshot was markedly less likely to follow simple formatting instructions than its March snapshot is squarely a behavioral drift finding, and it matters practically because many production applications parse LLM outputs programmatically (e.g., expecting JSON or a specific code block), such that a purely stylistic regression can break an otherwise-functioning downstream pipeline even without any change in the model's underlying reasoning ability. Sycophancy, as documented by Perez et al. (2023) and Sharma et al. (2023), can be viewed as a special case of behavioral drift in which the model's tendency to align its stated position with a user's expressed view shifts across versions, plausibly as a function of changes in the RLHF preference model or the human feedback data used to train it.

3.3 Alignment and Safety Drift

Alignment and safety drift refers to changes in a model's willingness to answer sensitive, harmful, or opinion-eliciting queries, and in the calibration between over-refusal (declining benign requests) and under-refusal (complying with genuinely harmful ones). Chen et al. (2024) found that GPT-4's June 2023 snapshot was more resistant to certain jailbreak-style prompts than its March snapshot — a change most observers would regard as an improvement — illustrating that safety drift is not inherently negative but is nonetheless drift in the sense that it changes system behavior in ways downstream integrators may not anticipate or have tested for. The alignment tax findings of Ouyang et al. (2022) suggest one structural source of this dimension: each round of RLHF fine-tuning is itself an optimization process with the potential to shift the model's behavior on benchmarks and use cases well outside the immediate target of the fine-tuning intervention, via mechanisms functionally related to the catastrophic forgetting documented by Kirkpatrick et al. (2017) in simpler sequential-learning settings.

3.4 Instance-Level Regression (Negative Flips)

The fourth dimension is not a distinct behavioral phenomenon so much as a distinct unit of measurement, but it deserves separate treatment because it is the dimension most directly borrowed

from, and validated against, the classical model-update literature. As formalized by Bansal et al. (2019) and adapted to LLMs by Echterhoff et al. (2024), instance-level regression asks not “did aggregate performance go up or down” but “which specific inputs, previously handled correctly, are now handled incorrectly.” This framing is important for LLM drift tracking because aggregate benchmark scores can mask substantial churn beneath the surface: it is entirely possible, and empirically documented, for a new model version to post a higher overall benchmark score while still introducing a meaningful rate of negative flips on a subset of previously reliable behavior — the exact pattern that erodes user trust even when a provider's release notes describe the update as a strict improvement.

3.5 Interaction Between Dimensions

These four dimensions are empirically entangled. A change in RLHF policy, for instance, can simultaneously alter instruction-following (behavioral drift), refusal rates (safety drift), sycophancy (also behavioral/alignment drift), and, through the instruction-following channel, apparent task accuracy on benchmarks that require a specific output format (capability drift, as measured in aggregate, though the underlying reasoning capability may be unchanged). This entanglement is precisely why Chen et al. (2024) hypothesized a single underlying mechanism — degraded instruction-following — as a partial explanation for drift observed across several nominally distinct tasks, and it is why drift measurement protocols that report only a single aggregate accuracy number per task are of limited diagnostic value: distinguishing a genuine capability regression from a formatting-driven scoring artifact requires either human-graded evaluation or measurement designs that separate content correctness from output-format compliance.

4. Current Approaches and Developments

4.1 Longitudinal and Continuous Benchmarking

The most direct response to LLM drift is periodic or continuous re-evaluation of a deployed model against a fixed task suite, as exemplified by Chen et al.'s (2024) two-snapshot comparison and, more systematically, by Tu et al.'s (2023) ChatLog, which automates monthly and daily response collection across 21 benchmarks specifically to “fit” a trajectory of model behavior over time rather than compare only two static points. This approach's central methodological requirement is a fixed, version-controlled evaluation harness — identical prompts, identical decoding parameters, and identical grading logic reused across every evaluation round — since without this control, apparent drift can be confounded with drift in the evaluation protocol itself.

4.2 Holistic and Multi-Metric Evaluation Frameworks

HELM (Liang et al., 2022) illustrates a complementary, breadth-oriented approach: rather than tracking a narrow set of tasks with high temporal resolution, HELM evaluates models across a wide taxonomy of scenarios and seven metrics (accuracy, calibration, robustness, fairness, bias, toxicity, and efficiency) under standardized prompting conditions, with results published for public comparison. Repeated HELM-style evaluation of the same nominal model across time — which the framework's authors explicitly envisioned by describing HELM as a “living benchmark” — offers a way to detect drift not just

in headline accuracy but in secondary properties such as calibration and toxicity, which the narrower task batteries used by Chen et al. (2024) and Tu et al. (2023) do not directly assess.

4.3 Backward-Compatible and Regression-Aware Training

A second family of approaches attempts to reduce drift at the training stage rather than merely detect it after deployment. Positive-congruent training (Yan et al., 2021) and backward-compatible weight interpolation (Schumann et al., 2024) both explicitly optimize a new model to preserve the correct predictions of its predecessor, trading a small amount of aggregate accuracy gain for a substantial reduction in negative flip rate. Applied specifically to LLMs, MUSCLE (Echterhoff et al., 2024) proposes a model-update strategy that reduces instance regression when base models are updated, evaluated across the Llama, Vicuna, and Phi model families on classification and generation tasks including HellaSwag and SAMsum. In the alignment domain, Ouyang et al.'s (2022) PPO-ptx approach — mixing pretraining-distribution log-likelihood objectives into the RLHF fine-tuning loss — functions as an analogous regression-aware training intervention aimed specifically at reducing alignment tax between the pre- and post-RLHF versions of a model.

4.4 Documentation and Transparency Mechanisms

Model cards (Mitchell et al., 2019) and the system cards subsequently adopted by several commercial LLM providers represent an institutional, rather than purely technical, response to drift: by publishing a structured record of a model's intended use, evaluation results, and known limitations at each release, providers create a documentary trail against which subsequent versions can, in principle, be compared. Mitchell et al. (2019) explicitly designed model cards to support exactly this kind of comparison across model versions, listing model details, evaluation data, and quantitative analyses as core sections. In practice, the granularity, task coverage, and update frequency of publicly released system cards vary considerably across providers, limiting their current usefulness as a drift-tracking instrument relative to their original design intent.

4.5 Contamination-Aware and Dynamic Evaluation

Because apparent capability improvement between versions can be inflated by benchmark data contamination — the inadvertent inclusion of evaluation data in training corpora (Xu et al., 2024) — recent evaluation research has moved toward dynamic benchmarks that are regenerated or time-stamped to reduce the likelihood that a given model version has seen the exact test items during training. This development is directly relevant to drift tracking because it addresses a key threat to validity: without contamination control, an apparent capability gain across versions may partly or wholly reflect the newer model having memorized more evaluation data, rather than a genuine improvement in the ability the benchmark is intended to measure, and the reverse concern applies with equal force to using benchmark score changes as evidence of genuine capability drift rather than fluctuating contamination levels.

5. Research Challenges and Limitations

5.1 Opacity of Commercial API Versioning

The most fundamental obstacle to drift research is that commercial LLM providers generally do not disclose the timing, scope, or technical content of updates to a given named model endpoint. Chen et al. (2024) explicitly note that “when and how these models are updated over time is opaque,” meaning that researchers can typically detect that drift has occurred between two dated snapshots but cannot attribute it to a specific change in training data, architecture, fine-tuning procedure, or system-level configuration such as a prompt template or safety filter. This opacity makes drift research fundamentally observational rather than experimental: without access to paired before/after model checkpoints and change logs, researchers cannot isolate causal mechanisms, only document correlational associations between calendar time and behavior change.

5.2 Benchmark Data Contamination

As discussed above, contamination confounds longitudinal comparison in both directions. Xu et al. (2024) document that benchmark contamination is pervasive and increasing as training corpora grow, and Touvron et al.'s widely cited finding that a substantial fraction of MMLU test items overlap with common pretraining corpora illustrates the scale of the problem. For drift researchers, this means that a benchmark suite validated as contamination-free against one model version may become contaminated for a later version trained on more recent web data, silently biasing apparent inter-version comparisons in the direction of overstated improvement.

5.3 Sampling Non-Determinism and Measurement Noise

LLM outputs are stochastic under non-zero decoding temperature, and even ostensibly deterministic settings can exhibit run-to-run variation in production API deployments due to hardware and batching non-determinism. Chen et al. (2024) address this partly through a bootstrapping approach that leverages GPT-4's temperature parameter to estimate answer-consistency and self-correction behavior, but the broader implication is that any single-sample comparison between two model versions risks conflating genuine drift with ordinary sampling variance, particularly for tasks near a model's competence boundary where small prompt or sampling perturbations can flip a response between correct and incorrect.

5.4 Absence of Standardized Drift Metrics

While the classical model-regression literature has converged on negative flip rate as a standard unit of analysis (Bansal et al., 2019; Yan et al., 2021), LLM drift studies to date have used heterogeneous, largely study-specific metrics — raw accuracy differences (Chen et al., 2024), rolling benchmark scores (Tu et al., 2023), or task-specific negative flip rates computed only for the narrower set of studies explicitly modeled on the classical regression literature (Echterhoff et al., 2024). The absence of a shared, standardized drift-reporting metric analogous to negative flip rate, applied consistently across capability, behavioral, and safety dimensions, makes it difficult to compare drift magnitude across studies, providers, or model families, or to establish acceptable drift thresholds analogous to regression-testing tolerances in conventional software engineering.

5.5 Cost and Scale of Continuous Evaluation

Comprehensive, HELM-style holistic evaluation is computationally and financially expensive — Liang et al. (2022) report costs on the order of tens of thousands of dollars for commercial model API calls in a single evaluation round covering dozens of models. Sustaining this level of evaluation on a continuous, per-update basis for every actively used commercial model, as would be required for reliable drift tracking, represents a nontrivial and largely unfunded infrastructure burden, one that currently falls unevenly on individual academic research groups (as in Chen et al., 2024, and Tu et al., 2023) rather than being systematically underwritten by model providers or a shared community resource.

6. Research Gaps and Future Directions

6.1 A Standardized, Multi-Dimensional Drift-Reporting Metric

Future work should extend the negative-flip-rate paradigm, already validated for capability drift (Bansal et al., 2019; Echterhoff et al., 2024), into a standardized, multi-dimensional drift report that separately quantifies capability regression, instruction-following or formatting compliance, refusal-rate change, and sycophancy shift between any two named model versions. Such a metric suite, published alongside each model release in a manner analogous to a model card (Mitchell et al., 2019), would allow downstream developers to assess update risk to their specific use case rather than relying on aggregate, task-agnostic release notes.

6.2 Causal Attribution of Drift Sources

Existing longitudinal studies are, as noted, observational: they document that drift occurred between two calendar dates but cannot attribute it to pretraining data changes, architecture changes, RLHF policy changes, or system-level prompt and safety-filter changes, because commercial providers do not expose paired checkpoints or change logs. Progress here will likely require either voluntary provider disclosure of update composition (analogous to software changelogs), controlled ablation studies conducted on open-weight model families where such variables can be manipulated directly (as Echterhoff et al., 2024, do for Llama, Vicuna, and Phi), or the development of behavioral probes specifically designed to distinguish reasoning-capability change from instruction-following or output-format change, addressing the confound Chen et al. (2024) identify but do not fully resolve.

6.3 Contamination-Robust Longitudinal Benchmarks

Because static benchmarks become progressively more contamination-prone as training corpora expand (Xu et al., 2024), drift-tracking infrastructure should adopt dynamically regenerated or strictly time-stamped test items, ensuring that any given evaluation item postdates the training cutoff of every model version under comparison. Combining this design principle with the rolling-evaluation cadence pioneered by ChatLog (Tu et al., 2023) would substantially strengthen the validity of longitudinal comparisons relative to either approach in isolation.

6.4 Community and Regulatory Infrastructure for Continuous Evaluation

Given the cost of comprehensive holistic evaluation documented by Liang et al. (2022), a shared, ideally provider-subsidized or publicly funded, evaluation-as-infrastructure resource — combining HELM's breadth of scenarios and metrics with ChatLog's temporal resolution and Chen et al.'s (2024) targeted, edge-case task design — would reduce duplicated effort across academic groups and make continuous drift monitoring financially sustainable. Such infrastructure would also provide a natural evidentiary basis for emerging regulatory reporting requirements for foundation models, which increasingly call for post-deployment performance monitoring in addition to pre-deployment evaluation.

6.5 Integrating Regression-Aware Training into Standard LLM Update Pipelines

Backward-compatible training methods such as positive-congruent training (Yan et al., 2021), backward-compatible weight interpolation (Schumann et al., 2024), and MUSCLE (Echterhoff et al., 2024) have so far been validated primarily in research settings on open-weight models. A significant open question is how such regression-aware objectives interact with RLHF and safety fine-tuning at the scale and update cadence used by commercial LLM providers, and whether reduction in negative flip rate can be achieved without reintroducing the alignment tax dynamics documented by Ouyang et al. (2022) — that is, whether backward compatibility with a prior version's task performance and alignment with current safety objectives can be jointly optimized rather than traded off against one another.

7. Conclusion

Large language models deployed under a fixed public name are not fixed artifacts: empirical longitudinal audits show that their capabilities, communication style, and safety behavior can change substantially across successive, often undisclosed, version updates (Chen et al., 2024; Tu et al., 2023). This paper has argued that LLM performance drift is best understood not as a single phenomenon but as a family of related effects — capability drift, behavioral and stylistic drift, alignment and safety drift, and instance-level regression — each traceable to distinct but interacting mechanisms rooted in established literatures on concept drift (Gama et al., 2014), model-update regression (Bansal et al., 2019; Yan et al., 2021), catastrophic forgetting (Kirkpatrick et al., 2017), alignment tax (Ouyang et al., 2022), and sycophancy (Perez et al., 2023; Sharma et al., 2023). Current detection and mitigation approaches, from longitudinal benchmarking and holistic evaluation frameworks to backward-compatible training and model-card documentation, provide a foundation for drift tracking but remain fragmented, costly to sustain, and vulnerable to confounds including API opacity, benchmark contamination, and sampling non-determinism. Closing these gaps — through standardized, multi-dimensional drift metrics, causal attribution methods, contamination-robust longitudinal benchmarks, and shared evaluation infrastructure — would allow performance drift tracking to move from an ad hoc research exercise, undertaken episodically by individual academic groups, toward a routine, auditable component of responsible LLM deployment, comparable in status to regression testing in conventional software engineering.

References

- Bansal, G., Nushi, B., Kamar, E., Weld, D. S., Lasecki, W. S., & Horvitz, E. (2019). Updates in human-AI teams: Understanding and addressing the performance/compatibility tradeoff. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01), 2429–2437. <https://doi.org/10.1609/aaai.v33i01.33012429>
- Chen, L., Zaharia, M., & Zou, J. (2024). How is ChatGPT's behavior changing over time? *Harvard Data Science Review*, 6(2). arXiv preprint arXiv:2307.09009. <https://doi.org/10.48550/arXiv.2307.09009>
- Echterhoff, J., Faghri, F., Vemulapalli, R., Hu, T.-Y., Li, C.-L., Tuzel, O., & Pouransari, H. (2024). MUSCLE: A model update strategy for compatible LLM evolution. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. arXiv preprint arXiv:2407.09435. <https://doi.org/10.48550/arXiv.2407.09435>
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), Article 44. <https://doi.org/10.1145/2523813>
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526. <https://doi.org/10.1073/pnas.1611835114>
- Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., Newman, B., Yuan, B., Yan, B., Zhang, C., Cosgrove, C., Manning, C. D., Ré, C., Acosta-Navas, D., Hudson, D. A., ... Koreeda, Y. (2022). Holistic evaluation of language models. arXiv preprint arXiv:2211.09110. <https://doi.org/10.48550/arXiv.2211.09110>
- McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24, 109–165. [https://doi.org/10.1016/S0079-7421\(08\)60536-8](https://doi.org/10.1016/S0079-7421(08)60536-8)
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)* (pp. 220–229). Association for Computing Machinery. <https://doi.org/10.1145/3287560.3287596>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744. arXiv preprint arXiv:2203.02155. <https://doi.org/10.48550/arXiv.2203.02155>
- Perez, E., Ringer, S., Lukošiušaitė, K., Nguyen, K., Chen, E., Heiner, S., Pettit, C., Olsson, C., Kundu, S., Kadavath, S., Jones, A., Chen, A., Mann, B., Israel, B., Seethor, B., McKinnon, C., Olah, C., Yan, D., Amodei, D., ... Kaplan, J. (2023). Discovering language model behaviors with model-written evaluations. In *Findings of the Association for Computational Linguistics: ACL 2023* (pp. 13387–13434). Association for Computational Linguistics. arXiv preprint arXiv:2212.09251. <https://doi.org/10.48550/arXiv.2212.09251>

- Schumann, R., Mansimov, E., Lai, Y.-A., Pappas, N., Gao, X., & Zhang, Y. (2024). Backward compatibility during data updates by weight interpolation. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2024) (pp. 2545–2560). Association for Computational Linguistics. arXiv preprint arXiv:2301.10546. <https://doi.org/10.48550/arXiv.2301.10546>
- Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askeel, A., Bowman, S. R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S. R., Kravec, S., Maxwell, T., McCandlish, S., Ndousse, K., Rausch, O., Schiefer, N., Yan, D., Zhang, M., & Perez, E. (2023). Towards understanding sycophancy in language models. arXiv preprint arXiv:2310.13548. <https://doi.org/10.48550/arXiv.2310.13548>
- Tu, S., Li, C., Yu, J., Wang, X., Tang, J., & Hou, L. (2023). ChatLog: Carefully evaluating the evolution of ChatGPT across time. arXiv preprint arXiv:2304.14106. <https://doi.org/10.48550/arXiv.2304.14106>
- Wei, J., Huang, D., Lu, Y., Zhou, D., & Le, Q. V. (2023). Simple synthetic data reduces sycophancy in large language models. arXiv preprint arXiv:2308.03958. <https://doi.org/10.48550/arXiv.2308.03958>
- Xie, Y., Lai, Y.-A., Xiong, Y., Zhang, Y., & Soatto, S. (2021). Regression bugs are in your model! Measuring, reducing and analyzing regressions in NLP model updates. arXiv preprint arXiv:2105.03048. <https://doi.org/10.48550/arXiv.2105.03048>
- Xu, C., Guan, S., Greene, D., & Kechadi, M.-T. (2024). Benchmark data contamination of large language models: A survey. arXiv preprint arXiv:2406.04244. <https://doi.org/10.48550/arXiv.2406.04244>
- Yan, S., Xiong, Y., Kundu, K., Yang, S., Deng, S., Wang, M., Xia, W., & Soatto, S. (2021). Positive-congruent training: Towards regression-free model updates. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2021) (pp. 14299–14308). IEEE. <https://doi.org/10.1109/CVPR46437.2021.01408>